অবজেক্ট ওরিয়েন্টেড প্রোগ্রামিং

দর্শন · স্তম্ভ · OOP বনাম POP · ক্লাস · অ্যাক্সেস · স্ট্যাটিক: ১২ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে ধারণার ওপর বাকি সব দাঁড়িয়ে

সিলেবাসের যে অংশ

OOP সিলেবাস শুরু হয় "**Philosophy of OOP; Advantages of OOP over structured programming; Encapsulation, classes and objects, access specifiers, static and non-static members**" দিয়ে। কনস্ট্রাক্টর, বিস্তারিত ইনহেরিট্যান্স, পলিমরফিজম ও টেমপ্লেট পরের দুই লেসনে।

হাই-ইল্ড তালিকা

① অবজেক্ট = আইডেন্টিটি + স্টেট + বিহেভিয়ার
② ক্লাস = ব্লুপ্রিন্ট; অবজেক্ট = ইনস্ট্যান্স
③ চার স্তম্ভ = A PIE
④ OOP বটম-আপ; POP টপ-ডাউন
⑤ এনক্যাপসুলেশন = বান্ডেল; হাইডিং = private
⑥ Java protected = প্যাকেজ + সাবক্লাস
⑦ C++ class → private, struct → public
⑧ static = ক্লাসপ্রতি একটি কপি
⑨ static মেথডে `this` নেই
⑩ Simula 67 = প্রথম OO ভাষা

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

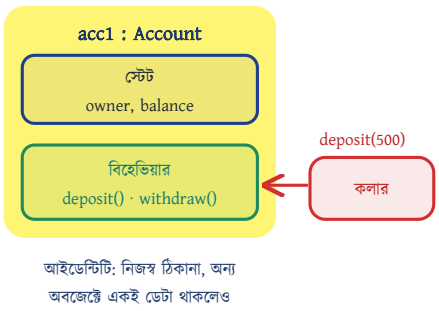
সূচিপত্র

✓ দর্শন: অবজেক্ট ও মেসেজ ... পৃ. ২
✓ চারটি স্তম্ভ ... পৃ. ৩
✓ অ্যাবস্ট্রাকশন বনাম এনক্যাপসুলেশন ... পৃ. ৪
✓ সুবিধা: OOP বনাম স্ট্রাকচার্ড ... পৃ. ৫
✓ ইতিহাস; পিওর বনাম হাইব্রিড ... পৃ. ৬
✓ ক্লাস ও অবজেক্ট ... পৃ. ৭
✓ অ্যাক্সেস স্পেসিফায়ার: Java ও C++ ... পৃ. ৮
✓ কোডে এনক্যাপসুলেশন: getter/setter ... পৃ. ৯
✓ স্ট্যাটিক বনাম নন-স্ট্যাটিক ... পৃ. ১০
✓ রাতের পরীক্ষার পাতা ... পৃ. ১১
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১২

১ OOP-এর দর্শন: অবজেক্ট ও মেসেজ

সংজ্ঞা: **অবজেক্ট ওরিয়েন্টেড প্রোগ্রামিং (OOP)** এমন একটি প্যারাডাইম যেখানে প্রোগ্রাম গড়া হয় **অবজেক্ট (Object)** দিয়ে — প্রতিটি অবজেক্ট নিজের ডেটা ও সেই ডেটার ওপর কাজ করার অপারেশন একসঙ্গে রাখে — এবং অবজেক্টগুলো একে অপরকে **মেসেজ পাঠিয়ে (Message passing)** কাজ করে। এক লাইনে দর্শন: *"বাস্তব জগৎকে যেভাবে ভাবি, সেভাবেই প্রোগ্রাম লেখো।"* গাড়ি, ছাত্র বা ব্যাংক অ্যাকাউন্টের কিছু **আছে** (স্টেট) এবং সেগুলো কিছু **করে** (বিহেভিয়ার); OOP ঠিক সেভাবেই লেখে।

ডেটা + কোড একসঙ্গে চলে



প্রতিটি অবজেক্টের তিন অংশ

অবজেক্টের তিনটি জিনিস (Booch): **আইডেন্টিটি (Identity)** — যা একে *এই* অবজেক্ট বানায়: একই owner ও balance-এর দুটি অ্যাকাউন্টও দুটি আলাদা অবজেক্ট (ভিন্ন ঠিকানা / রেফারেন্স)।

স্টেট (State) — অ্যাট্রিবিউটগুলোর (ফিল্ড, ডেটা মেম্বার) বর্তমান মান।

বিহেভিয়ার (Behaviour) — অবজেক্ট যা করতে পারে: তার মেথড (মেম্বার ফাংশন), যা স্টেট পড়ে বা বদলায়।

মেসেজ পাসিং — `acc1.deposit(500)` রিসিভার *acc1*-কে "deposit 500" মেসেজ পাঠায়: রিসিভার + মেথডের নাম + আর্গুমেন্ট। কীভাবে সাড়া দেবে, তা রিসিভারই ঠিক করে।

বাস্তব জিনিস	স্টেট (আছে)	বিহেভিয়ার (করে)	কোডে
গাড়ি	রং, ইঞ্জিন, গতি	চালু, থামা, গতি বাড়ানো	*Car*-এর ফিল্ড + মেথড
ছাত্র	নাম, আইডি, গ্রেড	পড়া, পরীক্ষা দেওয়া	*Student*-এর ফিল্ড + মেথড
ব্যাংক অ্যাকাউন্ট	মালিক, ব্যালেন্স	জমা, উত্তোলন	*Account*-এর ফিল্ড + মেথড

মনে রাখার কৌশল: **"বিশেষ্য হয় ক্লাস, বিশেষণ হয় ফিল্ড, ক্রিয়া হয় মেথড।"** সমস্যার বিবরণ জোরে পড়লেই প্রথম ক্লাস ডিজাইন হাতে আসে।

পরীক্ষার ফাঁদ: **আইডেন্টিটি আর স্টেট এক নয়।** সমান ফিল্ডের দুটি অবজেক্ট *সমান*, কিন্তু *অভিন্ন* নয়: Java-তে `a == b` আইডেন্টিটি, `a.equals(b)` স্টেট তুলনা করে।

সমাধানকৃত উদাহরণ ✍️ অংশ চেনো:
"লাইব্রেরি সদস্যদের বই ধার দেয়।" ক্লাস: **Library, Book, Member**। Book-এর স্টেট: title, isbn, onLoan। মেসেজ: `lib.lend(book, member)` → রিসিভার **lib**।

কোথা থেকে এল: শব্দটির প্রবর্তক অ্যালান কে (Alan Kay) OOP-কে বলেছিলেন "মেসেজিং, স্টেট নিজের কাছে রাখা ও লুকানো, এবং লেট বাইন্ডিং"। এই তিন ধারণাই নিচে ফিরে আসে **মেসেজ পাসিং**, **এনক্যাপসুলেশন** ও **পলিমরফিজম** নামে।

২ OOP-এর চারটি স্তম্ভ

সংজ্ঞা: চারটি নীতি একটি ভাষাকে অবজেক্ট ওরিয়েন্টেড করে: **অ্যাবস্ট্রাকশন (Abstraction)** (*কী* করে দেখাও, *কীভাবে* লুকাও), **এনক্যাপসুলেশন (Encapsulation)** (ডেটা ও তার কোড একসঙ্গে বেঁধে অ্যাক্সেস নিয়ন্ত্রণ), **ইনহেরিট্যান্স (Inheritance)** (নতুন ক্লাস পুরোনো ক্লাসের মেম্বার পায়) ও **পলিমরফিজম (Polymorphism)** ("এক ইন্টারফেস, অনেক রূপ")। ক্লাস ও অবজেক্ট হলো **ইট**; স্তম্ভগুলো তার ওপর গড়া **নীতি**।



A PIE: ক্লাস-অবজেক্টের মেঝের ওপর চার স্তম্ভ

স্তম্ভ	এক লাইনে অর্থ	Java / C++-এ উপায়	দৈনন্দিন ছবি
অ্যাবস্ট্রাকশন	জরুরি বৈশিষ্ট্য দেখাও, বাস্তবায়ন লুকাও	abstract class, interface, public মেথড	স্টিয়ারিং + ব্রেক, ইঞ্জিন লুকানো
এনক্যাপসুলেশন	ডেটা + মেথড এক ইউনিটে; অ্যাক্সেস নিয়ন্ত্রিত	class + `private` ফিল্ড + মেথড	ওষুধের ক্যাপসুল
ইনহেরিট্যান্স	চাইল্ড ক্লাস প্যারেন্টকে নেয় ও বাড়ায়	`extends` (Java), `: public` (C++)	Car is-a Vehicle
পলিমরফিজম	একই কল, টাইপভেদে ভিন্ন আচরণ	ওভারলোডিং (কম্পাইল টাইম), ওভাররাইডিং (রান টাইম)	বৃত্ত বনাম বর্গে *draw()*

স্তম্ভ পর্যায়ে ইনহেরিট্যান্স: একটি ক্লাস **বেস** ক্লাস থেকে **ডিরাইভড** হয়ে তার মেম্বার পায়: **is-a** সম্পর্ক (Dog is-a Animal); এর বিপরীতে **has-a** = কম্পোজিশন (Car has-a Engine)। প্রকারভেদ ও কনস্ট্রাক্টরের ক্রম: পরের লেসন। Java-তে একটি ক্লাস কেবল **একটি** ক্লাস `extends` করে, কিন্তু অনেক interface `implements` করতে পারে।

স্তম্ভ পর্যায়ে পলিমরফিজম: **কম্পাইল-টাইম** (স্ট্যাটিক বাইন্ডিং): ফাংশন / অপারেটর **ওভারলোডিং** — আর্গুমেন্ট তালিকা দেখে বাছাই। **রান-টাইম** (ডাইনামিক বাইন্ডিং): বেস-ক্লাস রেফারেন্স বা পয়েন্টারের মাধ্যমে **ওভাররাইডিং**; C++-এ ফাংশনটিকে `virtual` হতে হয়। বিস্তারিত: পরের একটি লেসন।

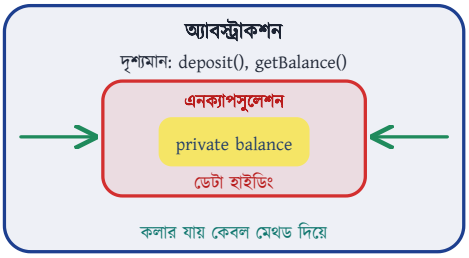
মনে রাখার কৌশল: A PIE = Abstraction, Polymorphism, Inheritance, Encapsulation। "কোনটি স্তম্ভ *নয়*?" — ওভারলোডিং, ক্লাস, অবজেক্ট, কম্পাইলেশন।

পরীক্ষার ফাঁদ: কিছু বই মাত্র **তিনটি** স্তম্ভ ধরে (অ্যাবস্ট্রাকশন বাদে)। অপশন বাধ্য না করলে চারটিই নাও। **ক্লাস** ও **অবজেক্ট** স্তম্ভ নয়।

সমাধানকৃত উদাহরণ ✍️ স্তম্ভ চেনো: `private int pin;` → **এনক্যাপসুলেশন**। `Savings extends Account` → ইনহেরিট্যান্স। ভিন্ন প্যারামিটারের দুটি `add()` → পলিমরফিজম।

৩ অ্যাবস্ট্রাকশন বনাম এনক্যাপসুলেশন বনাম ডেটা হাইডিং

সংজ্ঞা: তিন শব্দ, তিন কাজ। **অ্যাবস্ট্রাকশন** একটি **ডিজাইন** ধারণা: বাইরের জগতের কোন বৈশিষ্ট্য দরকার তা ঠিক করো, বাকিটা উপেক্ষা করো। **এনক্যাপসুলেশন** একটি **বাস্তবায়ন** কৌশল: ডেটা ও তা ব্যবহারকারী মেথড এক ক্লাসে মোড়ানো। **ডেটা হাইডিং (Data hiding)** এনক্যাপসুলেশনের ভেতরের **সুরক্ষা** ধাপ: ডেটাকে `private` করা, যাতে কেবল ক্লাসের নিজের মেথডই তা ছুঁতে পারে।



বাইরে → ইন্টারফেস → ক্যাপসুল → লুকানো ডেটা

একটি ক্লাসের তিন স্তর

বিষয়	অ্যাবস্ট্রাকশন	এনক্যাপসুলেশন
প্রশ্ন	**কী** করে?	**কীভাবে** মোড়ানো ও পাহারা?
স্তর	ডিজাইন	বাস্তবায়ন
লুকায়	জটিলতা	ডেটা (অ্যাক্সেস নিয়ন্ত্রণে)
উপায়	abstract class, interface	class + private + getter/setter
উদাহরণ	ATM দেখায় "withdraw"	*Account*-এ *balance* private

অ্যাবস্ট্রাক্ট ক্লাস বনাম ইন্টারফেস (স্তম্ভ পর্যায়ে)

বিষয়	Abstract class	Interface (Java)
অবজেক্ট তৈরি	ইনস্ট্যানশিয়েট করা যায় না	ইনস্ট্যানশিয়েট করা যায় না
মেথড	abstract ও concrete	abstract; Java 8 থেকে `default`/`static` বডি
ফিল্ড	যেকোনো ধরন, যেকোনো অ্যাক্সেস	কেবল `public static final` কনস্ট্যান্ট
কনস্ট্রাক্টর	আছে (সাবক্লাস চালায়)	নেই
একটি ক্লাস নিতে পারে	**একটি** (`extends`)	**অনেক** (`implements`)
C++-এ	≥ 1 pure virtual ফাংশন = `0`	আলাদা কিওয়ার্ড নেই: কেবল pure virtual-এর ক্লাস

মনে রাখার কৌশল: "**অ্যাবস্ট্রাকশন মেনু, এনক্যাপসুলেশন রান্নাঘরের দরজা, ডেটা হাইডিং সেই দরজার তালা।**" খদ্দের মেনু দেখে অর্ডার দেয়, রান্নাঘরে ঢোকে না।

পরীক্ষার ফাঁদ: **এনক্যাপসুলেশন =/ ডেটা হাইডিং।** সব ফিল্ড public হলেও ক্লাসটি এনক্যাপসুলেটেড (বান্ডেল), কিন্তু কিছুই লুকায় না। আর `private` একটি **কম্পাইল-টাইম** চেক, নিরাপত্তা নয়: Java রিফ্লেকশন তা পড়তে পারে।

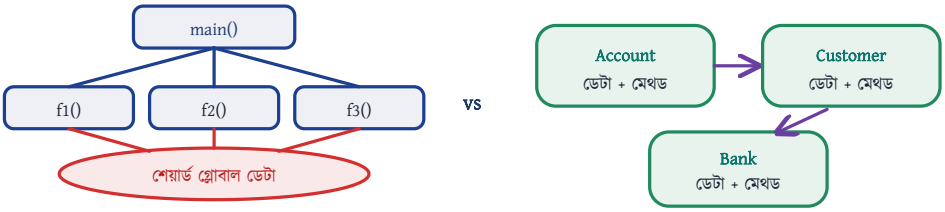
সমাধানকৃত উদাহরণ ✍️

শ্রেণিবিন্যাস:

"কেবল জরুরি বৈশিষ্ট্য দেখায়" → **অ্যাবস্ট্রাকশন**। "ডেটা ও ফাংশন এক ইউনিটে মোড়ায়" → **এনক্যাপসুলেশন**। "ফিল্ডে সরাসরি প্রবেশ ঠেকায়" → ডেটা হাইডিং।

৪ স্ট্রাকচার্ড প্রোগ্রামিংয়ের তুলনায় OOP-এর সুবিধা

সংজ্ঞা: **স্ট্রাকচার্ড (প্রসিডিউরাল) প্রোগ্রামিং** — C, Pascal, FORTRAN — সমস্যাকে **টপ-ডাউন (Top-down)** ভাবে ফাংশনে ভাগ করে, আর ডেটা ফাংশনগুলোর মধ্যে অবাধে ঘোরে, প্রায়ই গ্লোবাল হয়ে। **OOP** ক্লাস থেকে **বটম-আপ (Bottom-up)** গড়ে ওঠে, প্রতিটি ডেটা তার মালিক অবজেক্টের ভেতরে রেখে। সব সুবিধা এই এক পরিবর্তন থেকেই: **ডেটা থাকে তার কোডের সঙ্গে**।



বাঁয়ে: শেয়ার্ড ডেটার ওপর টপ-ডাউন ফাংশন · ডানে: মেসেজ বিনিময়কারী অবজেক্ট

বিষয়	স্ট্রাকচার্ড / প্রসিডিউরাল (POP)	অবজেক্ট ওরিয়েন্টেড (OOP)
ডিজাইন পদ্ধতি	**টপ-ডাউন**	**বটম-আপ**
প্রোগ্রামের একক	ফাংশন / প্রসিডিউর	ক্লাস ও অবজেক্ট
গুরুত্ব	অ্যালগরিদম (কী করতে হবে)	ডেটা (কী আছে)
ডেটা ও ফাংশন	আলাদা; গ্লোবাল ডেটা শেয়ার্ড	একসঙ্গে বাঁধা (এনক্যাপসুলেশন)
ডেটা নিরাপত্তা	কম: অ্যাক্সেস স্পেসিফায়ার নেই	বেশি: private / protected / public
পুনর্ব্যবহার	ফাংশন ও লাইব্রেরি	সঙ্গে ইনহেরিট্যান্স ও কম্পোজিশন
ফিচার যোগ	প্রায়ই পুরোনো ফাংশন বদলাতে হয়	সাবক্লাস যোগ; পুরোনো কোড অক্ষত
উপযোগী	ছোট প্রোগ্রাম, ড্রাইভার, ফার্মওয়্যার	বড়, দীর্ঘমেয়াদি সিস্টেম
উদাহরণ	C, Pascal, FORTRAN, COBOL	C++, Java, C#, Python, Smalltalk

লেখার মতো সাতটি সুবিধা: ① **ডেটা হাইডিং** · ② **পুনর্ব্যবহারযোগ্যতা** (ইনহেরিট্যান্স, কম্পোজিশন) · ③ **মডুলারিটি** — প্রতিটি ক্লাস আলাদা ডিবাগ · ④ **সহজ রক্ষণাবেক্ষণ** · ⑤ **সম্প্রসারণযোগ্যতা** — নতুন সাবক্লাস, পুরোনো কোড অক্ষত · ⑥ **নমনীয়তা** (পলিমরফিজম) · ⑦ **বাস্তব জগতের মডেল, স্কেলেবিলিটি**।

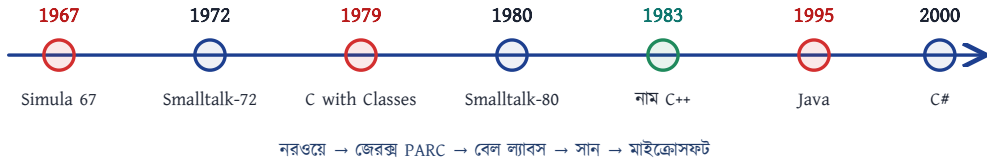
পরীক্ষার ফাঁদ: POP মানে "পুনর্ব্যবহার নেই" নয়: ফাংশন ও লাইব্রেরিও পুনর্ব্যবহার। OOP যোগ করে *সম্প্রসারণসহ* পুনর্ব্যবহার। OOP-এর খরচও আছে: বড় প্রোগ্রাম, বাড়তি ইনডিরেকশন, বেশি ডিজাইন শ্রম — ৫০ লাইনের স্ক্রিপ্টে সেরা নয়। ভালো ডিজাইন = **লো কাপলিং, হাই কোহেশন**।

মনে রাখার কৌশল: **"টপ-ডাউন কাজকে কাটে; বটম-আপ জিনিস গড়ে।"** কাপলিং (Coupling) = ক্লাসগুলো একে অপরের ওপর কতটা নির্ভর করে (কম রাখো); কোহেশন (Cohesion) = একটি ক্লাস কতটা একমুখী (বেশি রাখো)।

সমাধানকৃত উদাহরণ ✍️ গ্লোবাল ডেটার দাম: C-তে একটি পে-রোল *salary[]* গ্লোবাল রাখে; ৩০টি ফাংশন তা ছোঁয়। একটি বেতন ঋণাত্মক হলো: **৩০টি ফাংশনই** সন্দেহভাজন। OOP-তে কেবল *Employee.setSalary()* তা বদলাতে পারে, এবং ঋণাত্মক মান প্রত্যাখ্যান করে।

৫ OOP-এর ইতিহাস; পিওর বনাম হাইব্রিড ভাষা

সংজ্ঞা: OOP-এর শুরু নরওয়েতে **Simula 67** দিয়ে (ওলে-জোহান ডাল ও ক্রিস্টেন নিগার্ড), ক্লাস, অবজেক্ট, ইনহেরিট্যান্স ও ভার্চুয়াল প্রসিডিউরসহ প্রথম ভাষা। জেরক্স PARC-এ **Smalltalk** (অ্যালান কে) "সবকিছুই অবজেক্ট"-কে নিয়ম বানায়। **C++** (বিয়ার্নে স্ট্রাউস্ট্রুপ, বেল ল্যাবস) ও **Java** (জেমস গসলিং, সান) ধারণাটিকে মূলধারায় আনে।



নরওয়ে → জেরক্স PARC → বেল ল্যাবস → সান → মাইক্রোসফট

পাঁচটি সালেই বেশির ভাগ MCQ

ভাষা	সাল	স্রষ্টা / স্থান	যে জন্য মনে রাখবে
Simula 67	1967	Dahl ও Nygaard, নরওয়েজিয়ান কম্পিউটিং সেন্টার	**প্রথম OO ভাষা**
Smalltalk	1972 (প্রকাশ: -80)	Alan Kay, Dan Ingalls, Adele Goldberg; Xerox PARC	পিওর OO; Kay "OOP" শব্দের প্রবর্তক
C++	1979 → 1983 → 1985	Bjarne Stroustrup, বেল ল্যাবস	"C with Classes" ১৯৮৩-তে C++ নাম; প্রকাশ ১৯৮৫
Eiffel	1986	Bertrand Meyer	ডিজাইন বাই কন্ট্রাক্ট
Java	1995	James Gosling, সান (১৯৯১-এ *Oak* নামে শুরু)	"write once, run anywhere", JVM
C#	2000	Anders Hejlsberg, মাইক্রোসফট	.NET

শ্রেণি	অর্থ	উদাহরণ
পিওর OO	সবকিছু, এমনকি সংখ্যা ও ক্লাসও, অবজেক্ট	Smalltalk, Ruby, Eiffel
হাইব্রিড / আংশিক OO	OO-র সঙ্গে প্রসিডিউরাল বৈশিষ্ট্য; প্রিমিটিভ অবজেক্ট নয়	C++, Java, Objective-C, Python (মাল্টি-প্যারাডাইম)
অবজেক্ট-বেসড	অবজেক্ট ও এনক্যাপসুলেশন আছে, **ইনহেরিট্যান্স নেই**	Ada 83, Visual Basic 6

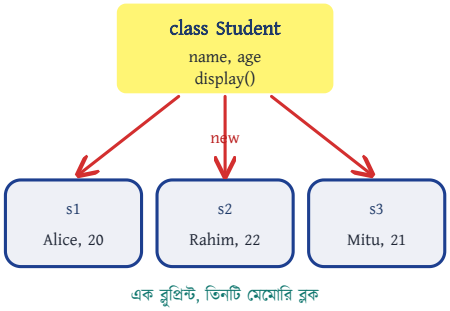
মনে রাখার কৌশল: "67 · 72 · 79/83 · 95": Simula, Smalltalk, C++ (জন্ম, নামকরণ), Java। স্ট্রাউস্ট্রুপের ভাষা চার বছর "C with Classes" ছিল।

পরীক্ষার ফাঁদ: Java পিওর OO নয়: `int`, `char` ইত্যাদি প্রিমিটিভ, আর `static` মেম্বার অবজেক্টের বাইরে থাকে। C++ **হাইব্রিড:** ক্লাস ছাড়াও বৈধ C++ প্রোগ্রাম লেখা যায়।

সমাধানকৃত উদাহরণ ✍️ সূত্র ধরে: "ক্লাস ও ইনহেরিট্যান্স সমর্থনকারী প্রথম ভাষা, সিমুলেশনের জন্য তৈরি" → **Simula 67**। "বেল ল্যাবস, ১৯৮৩" → C++।

৬ ক্লাস ও অবজেক্ট

সংজ্ঞা: **ক্লাস (Class)** হলো ইউজার-ডিফাইন্ড টাইপ: একটি **ব্লুপ্রিন্ট**, যা ডেটা মেম্বার (অ্যাট্রিবিউট) ও মেম্বার ফাংশন (মেথড) ঘোষণা করে। **অবজেক্ট** হলো ক্লাসের একটি **ইনস্ট্যান্স (Instance)** — মেমোরির একটি অংশ, যেখানে ওই অ্যাট্রিবিউটগুলোর এক সেট থাকে। ক্লাস **লজিক্যাল** সত্তা; অবজেক্ট **ফিজিক্যাল**। ইনস্ট্যান্স ডেটার মেমোরি বরাদ্দ হয় **প্রতিটি অবজেক্টে**, ক্লাস ঘোষণার সময় নয়।



এক ব্লুপ্রিন্ট, তিনটি মেমোরি ব্লক

ড্যাশ = ডেটার মেমোরি নেই

```
class Student {
    String name;        // state
    int age;            // state
    void display() {    // behaviour
        System.out.println("Name: " + name
                           + ", Age: " + age);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // object
        s1.name = "Alice";
        s1.age = 20;
        s1.display(); // Name: Alice, Age: 20
    }
}
```

বিষয়	ক্লাস	অবজেক্ট
কী	ব্লুপ্রিন্ট, ইউজার-ডিফাইন্ড টাইপ	সেই টাইপের ইনস্ট্যান্স
সত্তা	লজিক্যাল	ফিজিক্যাল (মেমোরি নেয়)
কতবার	ঘোষণা **একবার**	তৈরি **বহুবার**
তৈরি হয়	`class` কিওয়ার্ডে	`new` (Java); `Student s;`-ও (C++)
ফিল্ডের মেমোরি	নেই (static মেম্বার বাদে)	প্রতিটি নন-স্ট্যাটিক ফিল্ডের এক কপি

মনে রাখার কৌশল: **"ক্লাস বাড়ির নকশা; অবজেক্ট বাড়িগুলো।"** নকশায় বাস করা যায় না; প্রতিটি বাড়ির আসবাব (ফিল্ড) আলাদা, ঘর (মেথড) একই।

পরীক্ষার ফাঁদ: Java-তে `Student s1;` কেবল একটি **রেফারেন্স** (null), **অবজেক্ট নয়**; অবজেক্ট বানায় `new`। C++-এ `Student s1;` অবজেক্ট **বানায়**। C++-এ ফাঁকা ক্লাসের `sizeof` = 1, কখনো 0 নয়।

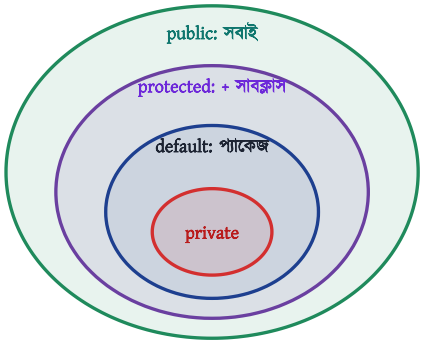
সমাধানকৃত উদাহরণ ✍️ অবজেক্ট গোনো:

`a = new Student(); b = a;` এবং `Student c;` → অবজেক্ট: **১টি**; রেফারেন্স: ৩টি।

একই ক্লাস C++-এ: `class Student { public: string name; int age; void display(); };` — `public:` না দিলে সব মেম্বার private। মেথড মেমোরিতে থাকে **একবারই**, সব অবজেক্ট ভাগ করে।

৭ অ্যাক্সেস স্পেসিফায়ার: Java ও C++

সংজ্ঞা: **অ্যাক্সেস স্পেসিফায়ার (Access specifier)** (Java-তে *access modifier*) ঠিক করে কে একটি মেম্বার ব্যবহার করতে পারবে। এভাবেই ক্লাস **ডেটা হাইডিং** কার্যকর করে। Java-তে **চারটি** স্তর — `private`, default (কিওয়ার্ড নেই, "package-private"), `protected`, `public`; C++-এ **তিনটি** কিওয়ার্ড — `private`, `protected`, `public`, সঙ্গে ব্যতিক্রম দিতে `friend`।



Java: প্রতিটি বলয় কলার বাড়ায়

Java মডিফায়ার	একই ক্লাস	একই প্যাকেজ	অন্য প্যাকেজের সাবক্লাস	অন্য প্যাকেজ
`private`	✓	✗	✗	✗
default (নেই)	✓	✓	✗	✗
`protected`	✓	✓	✓*	✗
`public`	✓	✓	✓	✓

* কেবল **ইনহেরিট্যান্সের মাধ্যমে** — সাবক্লাসের নিজের রেফারেন্স দিয়ে (`this` বা সাবক্লাস অবজেক্ট), সাধারণ প্যারেন্ট অবজেক্ট দিয়ে নয়।

C++ স্পেসিফায়ার	নিজ ক্লাস + friend	ডিরাইভড ক্লাস	বাইরে
`private`	✓	✗	✗
`protected`	✓	✓	✗
`public`	✓	✓	✓

কিছু না লিখলে ডিফল্ট	মেম্বার	ইনহেরিট্যান্স মোড
C++ `class`	**private**	private (`class D : B`)
C++ `struct`	**public**	public (`struct D : B`)
Java মেম্বার	package-private (default)	— (সবসময় `extends`)
Java টপ-লেভেল ক্লাস	কেবল `public` বা default চলে	প্রতি .java ফাইলে একটি `public` ক্লাস

মনে রাখার কৌশল: Java, সংকীর্ণ থেকে প্রশস্ত: "**Private Dogs Protect People**" = private → default → protected → public। C++: "**class গোপন রাখে, struct সব বলে দেয়।**"

পরীক্ষার ফাঁদ: **Java default ≠ C++ default।** Java default = প্যাকেজ; C++ class default = private। Java `protected` default-এর চেয়ে **প্রশস্ত** (অন্য প্যাকেজের সাবক্লাস যোগ হয়)। `static`, `final`, `friend` অ্যাক্সেস স্পেসিফায়ার **নয়**।

সমাধানকৃত উদাহরণ ✍️ x কে পড়তে পারে?:

```
package a; public class P { protected int x; }
```

a-র ক্লাস *Q*: **পারে**। প্যাকেজ *b*-তে R `extends P`: `this.x` হিসেবে পারে; `new P().x` দিয়ে নয়। *b*-র সাবক্লাস নয় এমন *S*: পারে না।

৮ কোডে এনক্যাপসুলেশন: Getter, Setter, যাচাই

সংজ্ঞা: **এনক্যাপসুলেশন** = ডেটা ও তার ওপর কাজ করা মেথড এক ক্লাসে বাঁধা এবং ডেটায় সরাসরি প্রবেশ সীমিত করা। প্রচলিত রেসিপি: প্রতিটি ফিল্ড `private`, পড়ার জন্য `public` **getter** (accessor), বদলানোর জন্য **setter** বা অপারেশন মেথড (mutator), আর তার ভেতরে **যাচাই (Validation)** — যাতে অবজেক্ট কখনো অবৈধ স্টেটে না থাকে।

```
class BankAccount {
  private String owner;
  private double balance;  // hidden
  public BankAccount(String o, double b) {
    owner = o;  balance = b > 0 ? b : 0;
  }
  public void deposit(double amt) {
    if (amt > 0) balance += amt;
  }
  public boolean withdraw(double amt) {
    if (amt <= 0 || amt > balance)
      return false;       // refused
    balance -= amt;  return true;
  }
  public double getBalance() { return balance; }
  public String getOwner()   { return owner; }
  public void setOwner(String o) {
    if (o != null && !o.isEmpty()) owner = o;
  }
}
```

সমাধানকৃত উদাহরণ ✍️ ব্যালেন্স ট্রেস: `BankAccount a = new BankAccount("Rahim", 1000);`
`a.deposit(500);` → 1500
`a.withdraw(2000);` → **false**, এখনো 1500
`a.withdraw(300);` → 1200
`a.deposit(-50);` → উপেক্ষিত, 1200
`a.balance = 9999;` → **কম্পাইল এরর**: balance private
শেষে `a.getBalance()` = **1200.0**

এনক্যাপসুলেশনে যা পাই: **অখণ্ডতা** — ঋণাত্মক ব্যালেন্স কখনো হয় না। **রিড-অনলি ফিল্ড** — getter আছে, setter নেই। **বদলানোর স্বাধীনতা** — কাল balance পয়সায় `long` হিসেবে রাখলেও `getBalance()`-এর কলার টের পায় না। **ডিবাগের এক জায়গা** — সব পরিবর্তন তিনটি মেথড দিয়ে যায়।

মেম্বার	অ্যাক্সেস	ধরন	কেন
`balance`, `owner`	`private`	ফিল্ড	ডেটা হাইডিং
`getBalance()`, `getOwner()`	`public`	getter / accessor	ফিল্ড উন্মুক্ত না করে পড়া
`deposit()`, `withdraw()`, `setOwner()`	`public`	mutator	যাচাইয়ের পরেই বদল

মনে রাখার কৌশল: "**ফিল্ড private, মেথড public, যাচাই ভেতরে।**" প্রতিটি এনক্যাপসুলেশন প্রোগ্রামের তিন শব্দ।

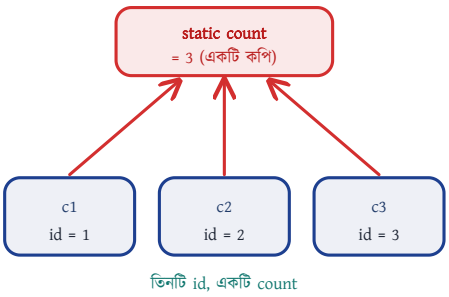
পরীক্ষার ফাঁদ: **যাচাইহীন** public setter নামেই এনক্যাপসুলেশন। `private` **ক্লাস**ভিত্তিক, অবজেক্টভিত্তিক নয়: Account-এর একটি মেথড অন্য Account-এর balance পড়তে পারে।

C++ সংস্করণ: একই নকশা: `class BankAccount { double balance; public: ... };` — *balance* **ডিফল্টভাবে** private; getter-টি `const`।

৯ স্ট্যাটিক বনাম নন-স্ট্যাটিক মেম্বার

সংজ্ঞা: **স্ট্যাটিক (static)** মেম্বার **ক্লাসের**: একটিই কপি, সব অবজেক্ট ভাগ করে, কোনো অবজেক্ট না থাকলেও থাকে, ব্যবহার হয় `ClassName.member` হিসেবে। **নন-স্ট্যাটিক** (ইনস্ট্যান্স) মেম্বার প্রতিটি **অবজেক্টের**: প্রতিটি অবজেক্টের নিজস্ব কপি। **স্ট্যাটিক মেথডে** `this` নেই, তাই তা **ইনস্ট্যান্স মেম্বার ছুঁতে পারে না** সরাসরি।

```
class Counter {
    static int count = 0;  // one, shared
    int id;                // one per object
    Counter() {
        count++;
        id = count;
    }
}
// in main():
Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();
System.out.print(c1.id + " " + c3.id
                 + " " + Counter.count);
// Output: 1 3 3
```



সব তীর একই বাক্সে পৌঁছায়

ধাপ	count	c1.id	c2.id	c3.id
ক্লাস লোড (static init)	0	–	–	–
new Counter() → c1	1	1	–	–
new Counter() → c2	2	1	2	–
new Counter() → c3	**3**	1	2	3

বিষয়	স্ট্যাটিক	নন-স্ট্যাটিক (ইনস্ট্যান্স)
মালিক	ক্লাস	প্রতিটি অবজেক্ট
কপি	একটি	প্রতি অবজেক্টে একটি
তৈরি	একবার, ক্লাস লোড / প্রোগ্রাম শুরুতে	প্রতিটি new-এ
মেথড ব্যবহার করতে পারে	কেবল static মেম্বার (`this`, `super` নেই)	static **ও** ইনস্ট্যান্স মেম্বার
C++ `sizeof`	অবজেক্টে **গোনা হয় না**	গোনা হয়

মনে রাখার কৌশল: **"স্ট্যাটিক = ক্লাসের নোটিশ বোর্ড; ইনস্ট্যান্স = প্রতিটি ছাত্রের খাতা।"** `main` কেন static? কোনো অবজেক্ট তৈরির আগেই JVM তাকে ডাকে।

পরীক্ষার ফাঁদ: `static void f()`-এর ভেতরে `id++` বা `this.id` **কম্পাইল এরর**; `new Counter().id` চলে। C++-এ বাইরে একবার সংজ্ঞা লাগে: `int Counter::count = 0;` static ব্লক চলে **একবার**, যেকোনো কনস্ট্রাক্টরের আগে।

সমাধানকৃত উদাহরণ ✍️ sizeof: `class A { int x; static int y; void f(); };` 4-বাইট int ধরে: `sizeof(A)` = **4** — static y ও ফাংশন f অবজেক্টে থাকে না। ফাঁকা ক্লাস: 1।

পরীক্ষার আগের রাতের পাতা

তথ্য পাতা:

অবজেক্ট = আইডেন্টিটি + স্টেট + বিহেভিয়ার

ক্লাস = ব্লুপ্রিন্ট; অবজেক্ট = ইনস্ট্যান্স

স্তম্ভ: অ্যাবস্ট্রাকশন, এনক্যাপসুলেশন, ইনহেরিট্যান্স, পলিমরফিজম

মেসেজ = রিসিভার + মেথড + আর্গুমেন্ট

POP টপ-ডাউন; OOP বটম-আপ

লো কাপলিং, হাই কোহেশন

Simula 67 = প্রথম OO ভাষা (নরওয়ে)

Smalltalk = পিওর OO; Alan Kay

C++: Stroustrup, 1979 → নাম 1983

Java: Gosling, সান, 1995 (Oak)

Java: private · default · protected · public

C++: class → private, struct → public

static = ক্লাসপ্রতি একটি কপি

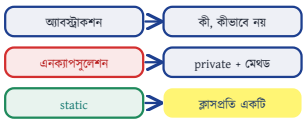
static মেথড: `this` নেই, ইনস্ট্যান্সে প্রবেশ নেই

MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. OOP-এর দর্শন: বাস্তব জগৎকে অবজেক্টে মডেল
2. এনক্যাপসুলেশন = ডেটা + ফাংশন এক ইউনিটে
3. অ্যাবস্ট্রাকশন = জরুরিটা দেখাও, বাস্তবায়ন লুকাও
4. ডেটা হাইডিং = ফিল্ড private করা
5. ইনহেরিট্যান্স = is-a; কম্পোজিশন = has-a
6. ওভারলোডিং = কম্পাইল টাইম; ওভাররাইডিং = রান টাইম
7. Java `protected` = প্যাকেজ + সাবক্লাস
8. Java default = package-private
9. Java: একটি `extends`, অনেক `implements`
10. মেমোরি প্রতি অবজেক্টে, প্রতি ক্লাসে নয়
11. C++ ফাঁকা ক্লাস: `sizeof` = 1
12. Java পিওর OO নয় (প্রিমিটিভ)

সেরা ফাঁদ: এনক্যাপসুলেশন =/ ডেটা হাইডিং · অ্যাবস্ট্রাকশন ডিজাইন, এনক্যাপসুলেশন বাস্তবায়ন · `static` অ্যাক্সেস স্পেসিফায়ার নয় · Java default ≠ C++ default · Java-তে `Student s;` অবজেক্ট বানায় না · static মেম্বার `sizeof`-এ নেই · POP-ও কোড পুনর্ব্যবহার করে।

সব কৌশল একসঙ্গে: বিশেষ্য → ক্লাস, ক্রিয়া → মেথড · A PIE · মেনু, রান্নাঘরের দরজা, তালা · টপ-ডাউন কাজ কাটে, বটম-আপ জিনিস গড়ে · 67 · 72 · 79/83 · 95 · বাড়ির নকশা বনাম বাড়ি · Private Dogs Protect People · নোটিশ বোর্ড বনাম খাতা।



তিন ধারণা, তিন লাইন

সমাধানকৃত উদাহরণ ✍️ একটি কিওয়ার্ডে আউটপুট বদল: *Counter* ক্লাসে *count* থেকে `static` সরাও: প্রতিটি অবজেক্টের count এখন 0 থেকে শুরু, তাই প্রতিটি *id* 1 এবং `c1.count` 1 — আউটপুট "1 3 3"-এর বদলে "**1 1 1**" (আর `Counter.count` আর কম্পাইল হয় না)।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. ডেটা ও তার ওপর কাজ করা ফাংশনকে এক ইউনিটে মোড়ানোকে বলে:
ক) ইনহেরিট্যান্স খ) পলিমরফিজম
গ) এনক্যাপসুলেশন ঘ) অ্যাবস্ট্রাকশন

২. অবজেক্ট হলো:
ক) একটি ক্লাসের ইনস্ট্যান্স খ) ক্লাস রিটার্ন করা ফাংশন
গ) একটি কিওয়ার্ড ঘ) একটি হেডার ফাইল

৩. প্রথম অবজেক্ট ওরিয়েন্টেড ভাষা কোনটি?
ক) C++ খ) Smalltalk
গ) Java ঘ) Simula 67

৪. Java-তে অ্যাক্সেস মডিফায়ারহীন মেম্বার দৃশ্যমান:
ক) কেবল নিজ ক্লাসে খ) নিজ প্যাকেজে
গ) সব জায়গার সাবক্লাসে ঘ) সব ক্লাসে

৫. C++-এ `class`-এর মেম্বার ডিফল্টভাবে:
ক) public খ) protected
গ) private ঘ) static

৬. কোন ধারণা কেবল জরুরি বৈশিষ্ট্য দেখায় ও বাস্তবায়ন লুকায়?
ক) অ্যাবস্ট্রাকশন খ) ইনহেরিট্যান্স
গ) ওভারলোডিং ঘ) পলিমরফিজম

৭. Java-তে static মেথড পারে না:
ক) ক্লাসের নামে কল হতে খ) static ফিল্ড ব্যবহার করতে
গ) public হতে ঘ) `this` ব্যবহার করতে

৮. OOP কোন ডিজাইন পদ্ধতি মানে?
ক) টপ-ডাউন খ) বটম-আপ
গ) বাম-ডান ঘ) কোনোটিই নয়

৯. কোনটি অ্যাক্সেস স্পেসিফায়ার নয়?
ক) private খ) public
গ) protected ঘ) static

১০. `class A { int x; static int y; };` 4-বাইট int হলে `sizeof(A)`:
ক) 8 খ) 4
গ) 0 ঘ) 1

উত্তরমালা → ১-গ · ২-ক · ৩-ঘ · ৪-খ · ৫-গ · ৬-ক · ৭-ঘ · ৮-খ · ৯-ঘ · ১০-খ

লিখিত প্রশ্ন: OOP-এর দর্শন ও চারটি স্তম্ভ লেখো। স্ট্রাকচার্ড প্রোগ্রামিংয়ের তুলনায় OOP-এর পাঁচটি সুবিধা দাও, এবং অ্যাক্সেস স্পেসিফায়ার ও একটি static মেম্বারসহ এনক্যাপসুলেশন দেখানো একটি ক্লাস লেখো। ১০ নম্বর

১ **দর্শন (২)** — সমস্যাকে আইডেন্টিটি, স্টেট ও বিহেভিয়ারসহ অবজেক্ট হিসেবে মডেল করা, যারা মেসেজ পাসিংয়ে সহযোগিতা করে; "বাস্তব জগৎকে যেভাবে ভাবি, সেভাবেই প্রোগ্রাম"। ২

২ **স্তম্ভ (২)** — অ্যাবস্ট্রাকশন (কী, কীভাবে নয়), এনক্যাপসুলেশন (বান্ডেল + পাহারা), ইনহেরিট্যান্স (is-a পুনর্ব্যবহার), পলিমরফিজম (এক ইন্টারফেস, অনেক রূপ)। ২

৩ **সুবিধা (৩)** — ডেটা হাইডিং, ইনহেরিট্যান্সে পুনর্ব্যবহার, মডুলারিটি ও সহজ রক্ষণাবেক্ষণ, পুরোনো কোড না বদলে সম্প্রসারণ, বাস্তব মডেল ও স্কেলেবিলিটি; POP টপ-ডাউন, শেয়ার্ড ডেটা। ৩

৪ **কোড (৩)** — `private` balance, যাচাইসহ `public` deposit/withdraw ও `getBalance()`, এবং কনস্ট্রাক্টরে বাড়ানো `static int count`; বলো count সবার, balance প্রতি অবজেক্টের। ৩

পরীক্ষার আগে ✓

- ✓ চার স্তম্ভের নাম বলতে পারি
- ✓ অ্যাবস্ট্রাকশন, এনক্যাপসুলেশন, হাইডিং আলাদা করতে পারি
- ✓ OOP বনাম POP টেবিল লিখতে পারি
- ✓ Java অ্যাক্সেস গ্রিড পূরণ করতে পারি
- ✓ static কাউন্টার ট্রেস করতে পারি
- ✓ Simula, C++, Java-র সাল বলতে পারি